

Heart Track Project – User Manual

Project Description

1. System Overview

Heart Track is an end-to-end IoT health monitoring project that collects heart rate (HR) and blood oxygen saturation (SpO2) from a MAX30105-based embedded device and presents the data in a web application. The system has three main components:

- a) Embedded device firmware (Arduino): Collects HR/SpO2, timestamps the measurement, and sends it to the backend.
- b) Backend API server (Node.js/Express + MongoDB): Authenticates users/physicians/devices, stores data, and provides daily/weekly query endpoints.
- c) Frontend web UI (static HTML + JavaScript): Provides a user portal for viewing personal metrics and managing devices, and a physician portal for viewing selected patients.

The backend serves the frontend as static assets from `project/public/`, so the entire application can be deployed as a single Node.js service.

2. Backend Implementation

- a) The backend is implemented as an Express application. It loads environment variables using `dotenv`, connects to MongoDB through `mongoose`, and enables common middleware such as JSON body parsing, `cors`, and request logging via `morgan`. Static files are served from `project/public/`.
An SPA-style catch-all route responds to any unmatched path by returning the user application page (`app.html`), which allows direct navigation/refresh without needing separate server-side rendering routes.

- b) Heart Track uses JSON Web Tokens (JWTs) for session authentication:
 - i. Users: Receive a JWT after `/api/auth/login` or `/api/auth/register`. The token contains `{id}` and is validated by `middleware/auth.js` on protected user endpoints.
 - ii. Physicians: Receive a JWT after `/api/physicians/login` or `/api/physicians/register`. The token contains `{id, role: "physician"}` and is validated by `protectPhysician`.
 - iii. Devices: Use an API key instead of a JWT. The device includes an `x-api-key` header that must match `DEVICE_API_KEY` configured on the server.

- c) The database schema is defined with Mongoose models:
 - i. User: Email/password identity, profile name, measurement configuration defaults (frequency/start/end), and an optional assigned physician.
 - ii. Physician: Login identity plus specialty/practice metadata, and a list of patients (also maintained through user assignment).
 - iii. Device: A physical device record with a unique `deviceId`, a user owner reference, and a friendly device name.
 - iv. Measurement: Individual data points linking to both the device and the

owning user, storing `heartRate`, `spo2`, and `timestamp`.

d) API endpoints

i. User authentication:

POST `/api/auth/register`: Create a user and register an initial device.

POST `/api/auth/login`: Authenticate and return JWT + user payload.

ii. User account management:

GET `/api/account/me`: Return current user profile, devices, and physician assignment info.

PUT `/api/account/me`: Update profile name and/or password.

PUT `/api/account/config`: Update measurement frequency and allowed time window.

GET/POST `/api/account/devices`: List or create user devices.

PUT/DELETE `/api/account/devices/:id`: Rename or delete a device (and its measurements).

GET/PUT/DELETE `/api/account/physician`: Get, assign, or clear the user's selected physician.

iii. Measurements:

POST `/api/measurements` (device API key required): Ingest HR/SpO2 readings. If the device is unknown, the server can auto-register it when `userId` is provided.

GET `/api/measurements/daily` (JWT required): Return a chart-friendly time series for a requested day.

GET `/api/measurements/weekly` (JWT required): Return aggregated weekly statistics (avg/min/max).

iv. Physician portal:

GET `/api/physicians/public`: Public directory of physicians (for user assignment).

GET `/api/physicians/me`: Physician profile.

GET `/api/physicians/patients`: Patients who assigned the physician, including weekly metrics and their configured frequency.

GET `/api/physicians/patients/:patientId/daily`: Daily time series for a specific patient and date.

PUT `/api/physicians/patients/:patientId/frequency`: Update the selected patient's configured measurement frequency.

e) Daily queries are performed using a day boundary defined in UTC (`startDate.setUTCHours(0,0,0,0)` and `< endDate`). The frontend sends a `YYYY-MM-DD` date string derived from the user's local date picker input.

3. Frontend Implementation (User Web App)

a) The user interface is delivered as static HTML and JavaScript. Tailwind CSS is used for styling, Chart.js is used for charts, and Lucide is used to render icons. The application behaves like a small single-page app: it shows/hides views and updates the DOM based on state.

b) When the user logs in or signs up:

i. The frontend calls `/api/auth/login` or `/api/auth/register`.

- ii. The JWT token is persisted to `localStorage` so sessions survive page refresh.
 - iii. On page load, if a stored token exists, the app auto-logs-in by calling `/api/account/me`.
- c) Weekly Summary tab
 - i. Loads `/api/measurements/weekly` once at app bootstrap and displays avg/min/max HR.
- d) Daily Detail tab
 - i. Uses a date input (`daily-date-picker`) whose default value is set to “today” (in local time).
 - ii. When the date changes, the frontend requests `/api/measurements/daily?date=YYYY-MM-DD`, updates its `dailyMetrics` state, and redraws the charts.
 - iii. The HR and SpO2 line charts are rendered with Chart.js; points corresponding to the minimum and maximum values are highlighted.
- e) Settings and physician assignment
 - i. Update profile name and/or password.
 - ii. Adjust measurement configuration (frequency and time-of-day range).
 - iii. Select a physician from a directory (loaded from `/api/physicians/public`) so the physician can view the user’s metrics in the physician portal.
- 4. Frontend Implementation (Physician Portal)
 - a) The physician portal has its own login/register forms and its own token storage key. After authentication, the portal loads the physician profile and the list of patients who have assigned that physician.
 - b) The “All Patients” view shows all assigned patients along with their 7-day average/min/max HR and configured frequency. Selecting a patient persists the selection in `localStorage` so the portal can restore it on refresh.
 - i. Weekly view:
 - Displays the selected patient’s weekly stats (calculated server-side via aggregation).
 - Allows updating the patient’s measurement frequency.
 - ii. Daily view:
 - Provides a date selector and two-line charts (HR and SpO2) for the selected patient.
 - On date changes, the portal requests `/api/physicians/patients/:patientId/daily?date=YYYY-MM-DD` and redraws the charts using the same Chart.js configuration patterns used by the user portal.
- 5. Embedded Device Implementation
 - a) The embedded component uses:
 - i. MAX30105 optical sensor to capture red/IR values.
 - ii. A provided SpO2/heart-rate algorithm implementation to compute HR and SpO2 from buffered samples.
 - iii. WiFi (Wi-FiS3) to connect to the network.
 - iv. ArduinoHttpClient to send HTTP POST requests to the backend API.

- v. RTC + NTPClient to keep time reasonably accurate.
- vi. Arduino LED Matrix to provide user feedback (prompt, success, failure icons).
- b) The device firmware uses a simple state machine:
 - i. IDLE: Waits until it is time to take a measurement (and within the allowed hours). The first run forces an initial measurement immediately.
 - ii. REQUESTING: Prompts the user to place a finger on the sensor (blinking heart icon) with a timeout. When a finger is detected (IR threshold), it transitions to measuring.
 - iii. MEASURING: Collects a fixed buffer of samples. If the finger is removed or data is unstable, it returns to REQUESTING. When enough samples are collected, it computes HR/SpO2 and attempts upload.
- c) Measurements are sent to the server's ingest endpoint:
 - i. Method/Path: POST `/api/measurements`
 - ii. Headers: `Content-Type: application/json`, `x-api-key: <DEVICE_API_KEY>`
 - iii. Body: Includes `deviceId`, `heartRate`, and `spo2` (and optionally `userId`/`deviceName` for auto-registration).
- d) If the device cannot upload (e.g., network down), it stores measurements in an offline queue (bounded circular buffer). When WiFi is available, it periodically retries uploading queued measurements.

File Description

1. Root Directory
 - a) README.md

This file describes how to set up and run the Node.js backend (Node version, npm install, and start commands). It also documents example user and physician credentials and password rules.
 - b) UserManual.txt

This document. It is intended to be copied into a formatted report and provides both an implementation narrative and per-file descriptions.
 - c) arduino.ino

The embedded firmware for the IoT device. It includes WiFi configuration, NTP + RTC time handling, MAX30105 sensor configuration, measurement computation, LED matrix feedback, HTTP upload to the backend API, and offline queuing logic.
2. `project/` (Backend Node.js application)
 - a) `project/app.js`

The main Express application entrypoint used by `npm start`. It loads environment variables, connects to MongoDB, registers middleware, serves static assets, mounts API route modules under `/api/*`, and finally provides a catch-all route that serves `public/app.html`.
 - b) `project/package.json`

Defines the Node.js package metadata and dependency list. It also provides

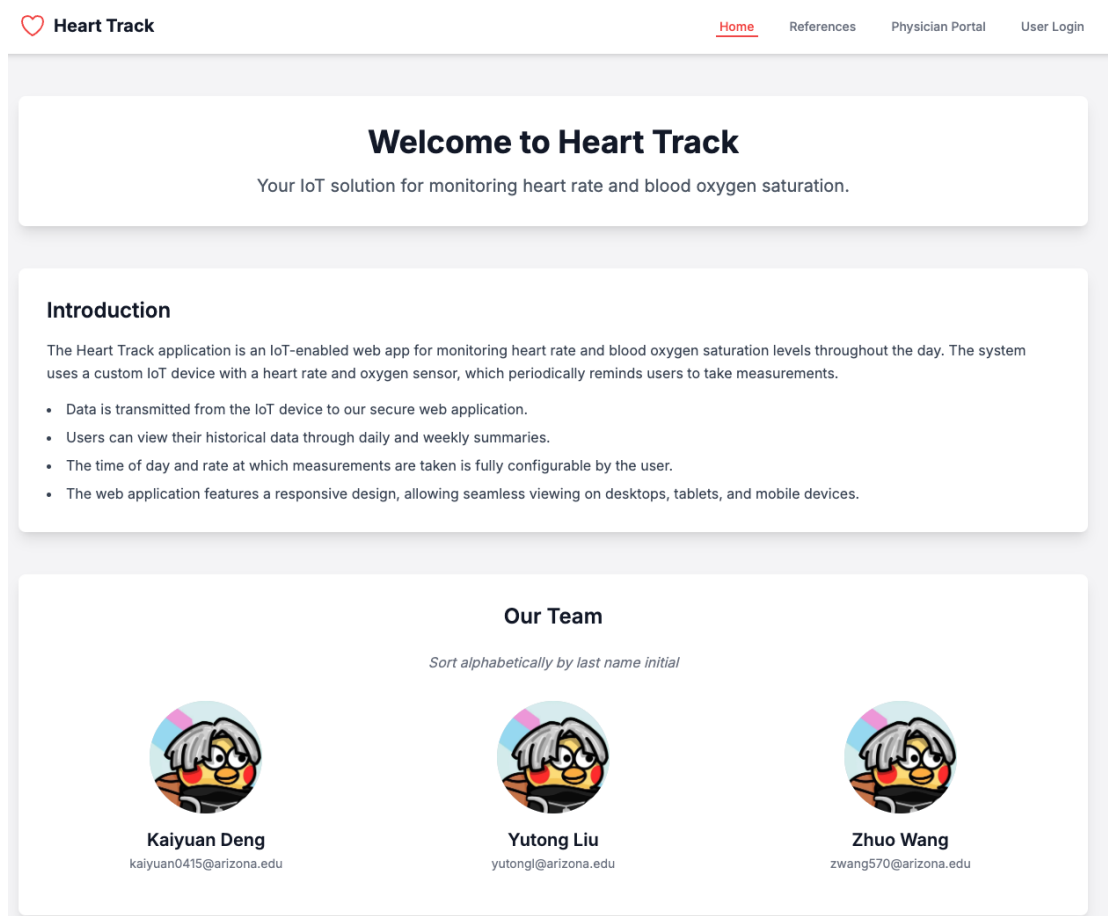
- the ``start`` and ``dev`` scripts for running the backend.
- c) `project/package-lock.json`
Locks the exact dependency versions for deterministic installs. This file is generated by npm and should typically not be edited manually.
 - d) `project/bin/www`
An Express-generator style HTTP server bootstrap script. In this project, the ``npm start`` script runs ``project/app.js`` directly, so ``bin/www`` is typically unused, but it remains as a standard scaffold.
 - e) `project/middleware/auth.js`
Implements authorization middleware for the API. It validates user JWTs (``protect``), physician JWTs with role checking (``protectPhysician``), and device API-key requests (``protectDevice``).
 - f) `project/models/User.js`
Mongoose model for user accounts. It defines the user schema (email, hashed password, display name, configuration, and physician assignment) and provides helper methods for password comparison and JWT issuance.
 - g) `project/models/Physician.js`
Mongoose model for physician accounts. It defines physician identity fields, password hashing, JWT issuance with a physician role claim, and a list of patients (users).
 - h) `project/models/Device.js`
Mongoose model for physical devices. Each device has a unique ``deviceId``, an owning user, and a friendly name.
 - i) `project/models/Measurement.js`
Mongoose model representing a single heart-rate/SpO2 measurement. Each measurement links to a user and device and stores numeric values plus a timestamp.
3. ``project/routes/`` (Backend API route modules)
- a) `project/routes/auth.js`
Implements user registration and login endpoints. It creates the initial device at registration time and returns a JWT plus a normalized user payload.
 - b) `project/routes/account.js`
Provides authenticated user account endpoints: profile read/update, measurement configuration updates, device management (CRUD), and physician assignment (assign/clear) while keeping physician-patient relationships synchronized.
 - c) `project/routes/measurements.js`
Provides measurement ingestion (device API key protected), daily time-series querying, and weekly summary aggregation. It formats daily results into labels and arrays suitable for Chart.js on the frontend.
 - d) `project/routes/physicians.js`
Implements physician registration/login, a public physician directory, physician profile endpoints, and physician access to assigned patients including weekly stats, daily metrics per patient, and patient frequency updates.

- e) `project/routes/helpers/userPayload.js`
Centralizes creation of the “user payload” returned to the frontend. It joins user profile data with devices and (optionally) physician info to provide a single response structure.
 - f) `project/routes/index.js`
A legacy Express-generator route that renders a Pug view. It is not used by the deployed static frontend but remains in the repository.
 - g) `project/routes/users.js`
Another legacy Express-generator scaffold route. It is not used by the Heart Track frontend and can be considered placeholder/demo code.
4. ``project/public/`` (Frontend static assets)
- a) `project/public/index.html`
The project home page. It provides an introduction to Heart Track and lists team members. It uses Tailwind CSS and Google Fonts for styling.
 - b) `project/public/app.html`
The user-facing application page containing the login/sign-up UI and the authenticated portal views (dashboard, devices, and settings). It loads Tailwind CSS, Chart.js, and Lucide for the UI.
 - c) `project/public/physician.html`
The physician portal page containing physician login/register UI and the authenticated portal views (patient list, weekly summary, and daily detail). It loads Tailwind CSS and Chart.js for layout and charting.
 - d) `project/public/reference.html`
The references page listing third-party libraries and services used in the project (frontend CDNs, backend packages, and embedded libraries/services).
 - e) `project/public/javascripts/app.js`
Client-side logic for the user portal. It manages JWT storage, performs API calls to ``/api/*``, maintains UI state (current user, devices, weekly/daily metrics), renders charts via Chart.js, handles the daily date picker, and implements device and settings interactions.
 - f) `project/public/javascripts/physician.js`
Client-side logic for the physician portal. It manages physician JWT storage, loads assigned patients and their metrics, persists patient selection, renders weekly and daily views, provides a daily date picker for patient data, and allows frequency updates.
 - g) `project/public/stylesheets/style.css`
A leftover stylesheet from the Express generator template. The main application UI is styled with Tailwind CSS, so this file is not central to the current UI.
 - h) `project/public/images/selfie.webp`
Local image asset used on the home page for team member photos.
5. ``project/views/`` (Legacy server-rendered templates)
- a) `project/views/layout.pug`
Defines a minimal Pug layout that includes ``stylesheets/style.css``. It is part of the Express-generator scaffold and not used by the static frontend flow.

- b) `project/views/index.pug`
Simple Pug “Welcome” page used by the legacy `routes/index.js`.
 - c) `project/views/error.pug`
Error template used by Express-generator scaffolding. The current application primarily responds with JSON for API errors and serves static HTML for the frontend.
6. `project/node_modules/` (Third-party dependencies)
- a) `project/node_modules/`
Contains installed third-party packages required by the Node.js backend. This directory is managed by npm and should not be edited by hand.

Result

1. Home page



2. References

References

Third-party APIs, libraries, and external code used in this project.

Frontend

- Tailwind CSS (loaded from CDN): <https://cdn.tailwindcss.com>
- Google Fonts — Inter: <https://fonts.google.com/specimen/Inter>
- Chart.js (charting library; loaded via jsDelivr): <https://www.chartjs.org/>
- Lucide (icon library; loaded via unpkg): <https://lucide.dev/>
- placeholder.co (placeholder avatar image): <https://placeholder.co/>

Backend

- Express (express): <https://expressjs.com/>
- Mongoose (mongoose): <https://mongoosejs.com/>
- MongoDB Node.js Driver (transitive dependency via Mongoose): <https://www.mongodb.com/docs/drivers/node/current/>
- JSON Web Tokens (jsonwebtoken): <https://github.com/auth0/node-jsonwebtoken>
- Password hashing (bcryptjs): <https://github.com/dcodeIO/bcrypt.js>
- CORS middleware (cors): <https://github.com/expressjs/cors>
- HTTP request logging (morgan): <https://github.com/expressjs/morgan>
- Debug utility (debug): <https://github.com/debug-js/debug>
- Environment variables (dotenv): <https://github.com/motdotla/dotenv>
- Development auto-reload (nodemon): <https://nodemon.io/>

3. User login page

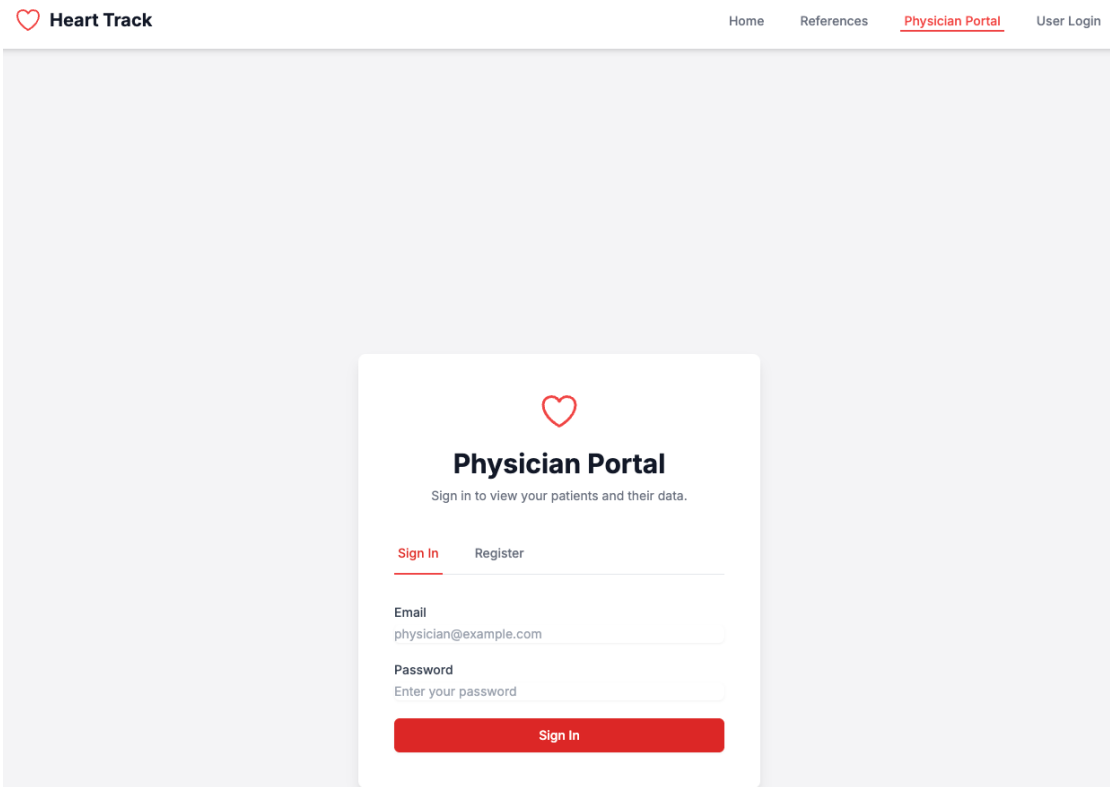


Login to Heart Track

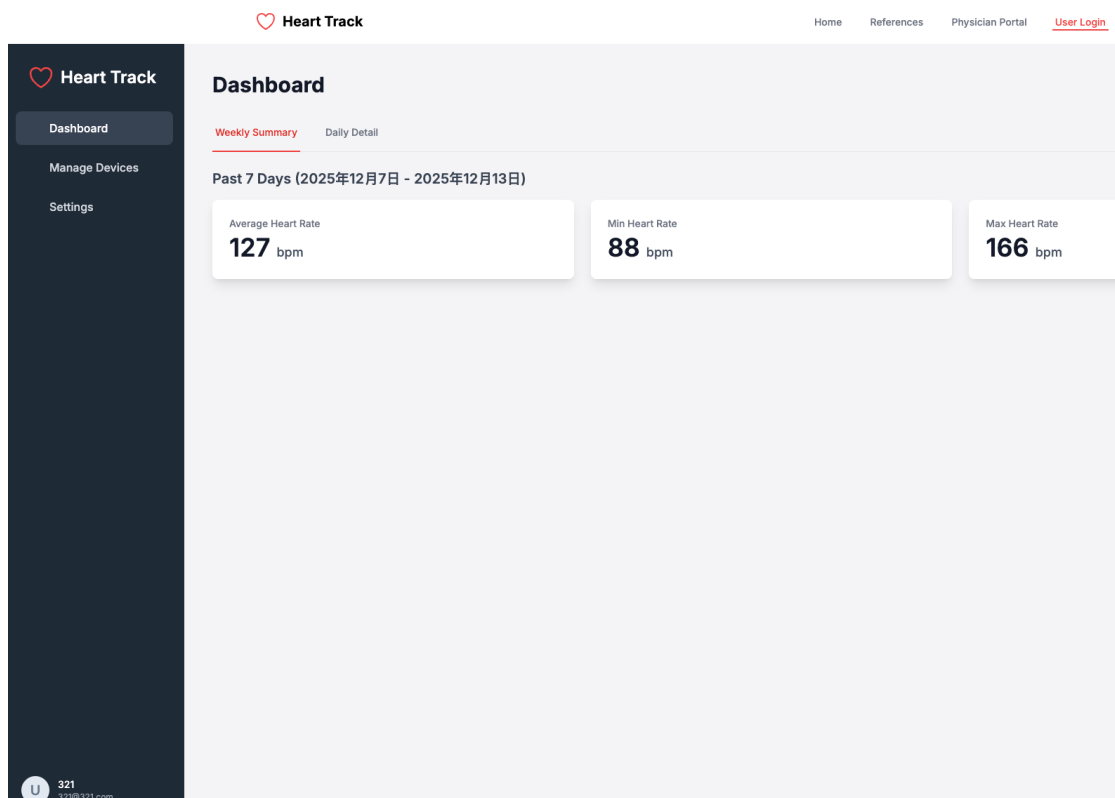
Or sign up if you don't have an account.

[Login](#) [Sign Up](#)

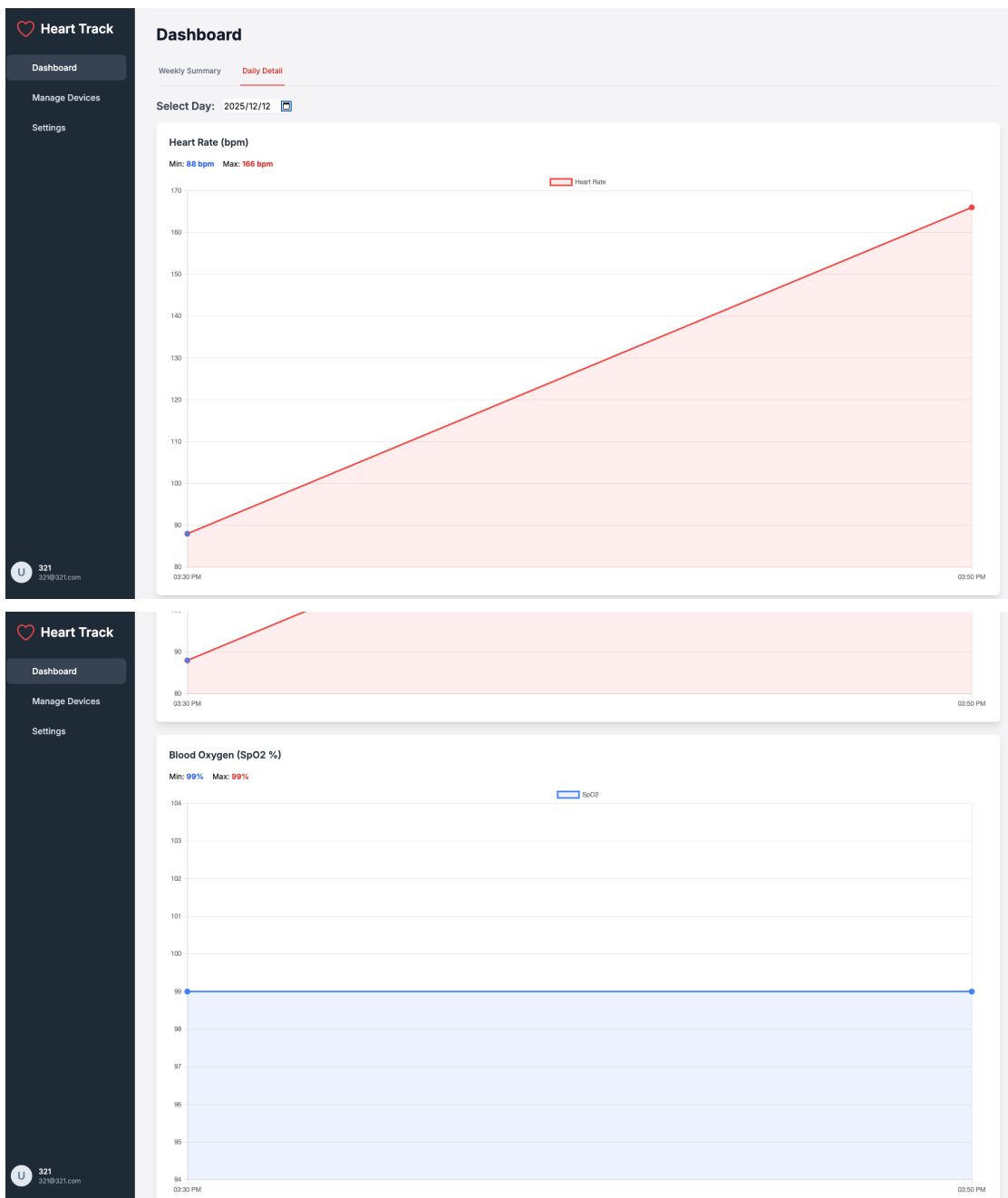
4. Physician login page



5. User weekly summary



6. User daily view



7. Device management page

Heart Track

Dashboard

Manage Devices

Settings

U

321

321@321.com

Manage Devices

Add New Device

Device Name

e.g., Bedroom Monitor

Device ID

Find this ID on your device

Add Device

Your Devices

Initial Device	1234 5678	Edit	Remove
111	5678 1234	Edit	Remove

8. Personal information setting page

Heart Track

Dashboard

Manage Devices

Settings

U

321

321@321.com

Settings

Account Information

Name

321

Email

321@321.com

New Password

Leave blank to keep unchanged

Confirm New Password

Confirm new password

Save Account

Measurement Configuration

Measurement Frequency

Every 30 minutes

Time-of-Day Range (Start)

08:00

Time-of-Day Range (End)

22:00

Save Configuration

Physician Assignment

Current physician: 123 (d1571502273@123.com)

Choose a registered physician

123 (d1571502273@123.com)

Save Physician

9. Physician home page

Heart Track MD

All Patients

Patient Summary

Patient Daily View

Heart Track

HomeReferencesPhysician PortalUser Login

Logged in as
123
d1571502273@123.com

CURRENT PATIENT
None selected

Patient List

All patients who have selected you as their physician.

1 patient

PATIENT	AVG (7D)	MAX (7D)	MIN (7D)	FREQUENCY	ACTION
321 321@321.com	127 bpm	166 bpm	88 bpm	30 min	View

10. Physician’s selected patient weekly view

Heart Track MD

All Patients

Patient Summary

Patient Daily View

Heart Track

HomeReferencesPhysician PortalUser Login

Logged in as
123
d1571502273@123.com

CURRENT PATIENT
321 (321@321.com)

321

321@321.com

AVERAGE HEART RATE
127 bpm

MAXIMUM HEART RATE
166 bpm

MINIMUM HEART RATE
88 bpm

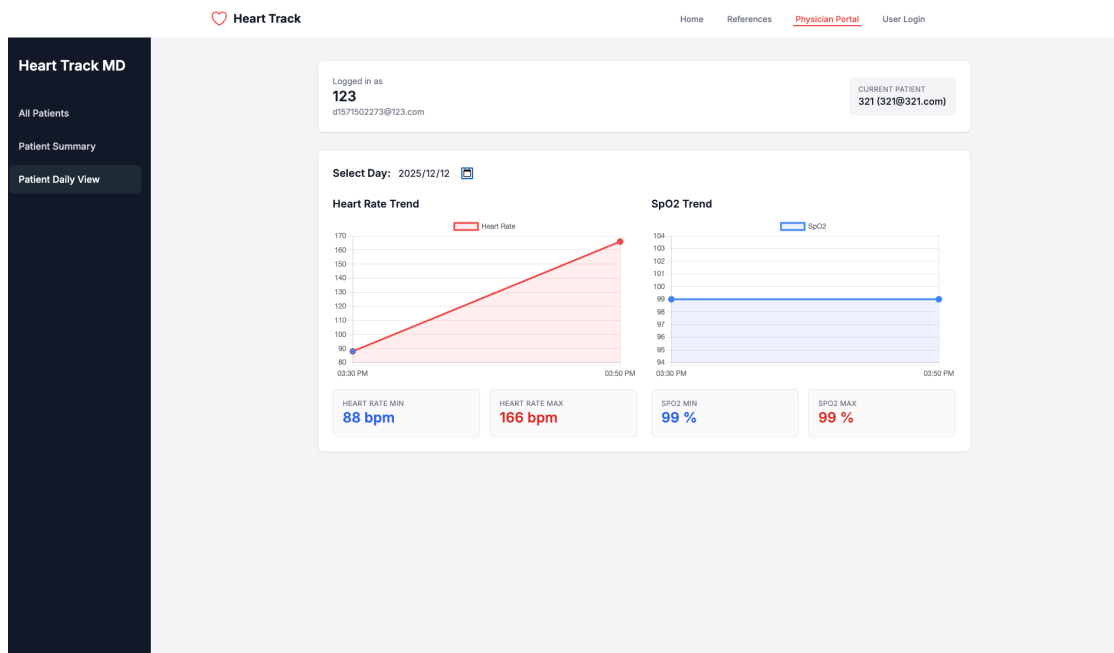
Measurement Schedule

Measurement Frequency
Every 30 minutes

Adjusting the frequency here updates the plan shown in your physician portal summary.

Update Frequency

11. Physician’s selected patient daily view



12. User signup page

Heart Track

Home References Physician Portal User Login

Login to Heart Track

Or sign up if you don't have an account.

Login Sign Up

Email address

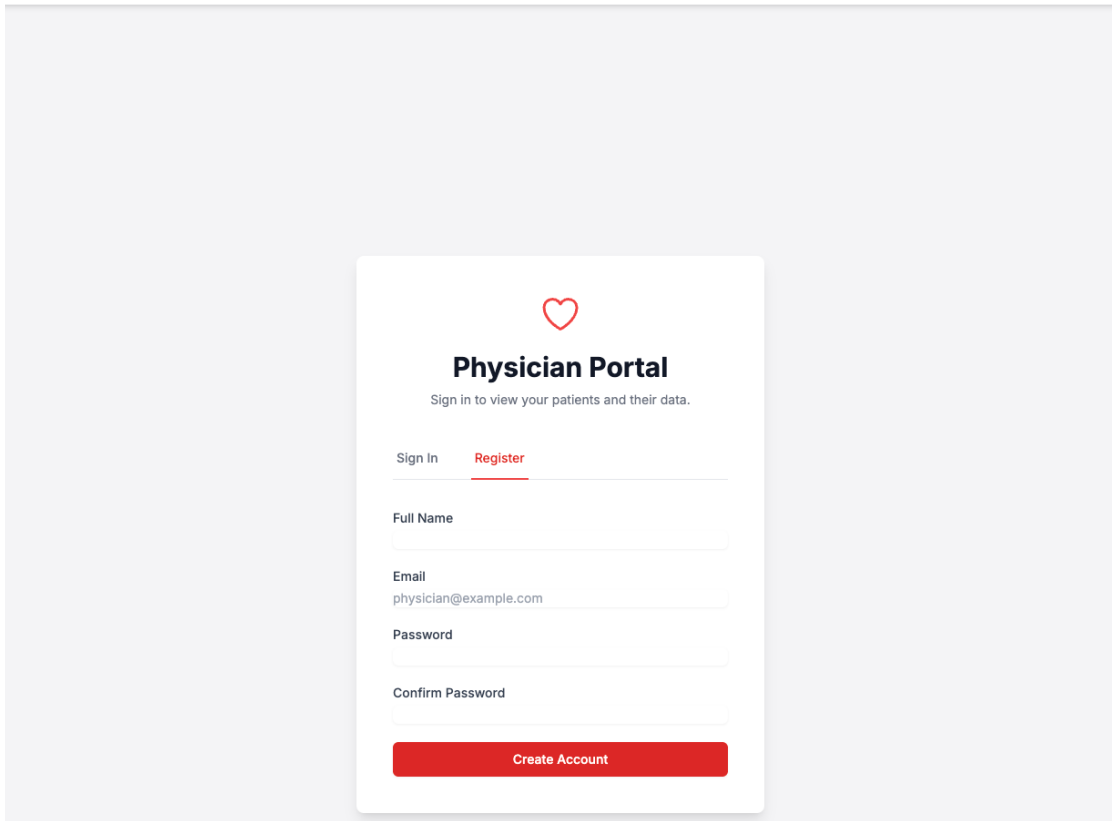
Initial Device ID (e.g., A1B2C3)

Password

Confirm Password

Sign Up

13. Physician signup page



The image shows a web browser window displaying the 'Physician Portal' registration page. The page has a light gray background. At the top, there is a navigation bar with the 'Heart Track' logo on the left and links for 'Home', 'References', 'Physician Portal' (which is underlined and highlighted in red), and 'User Login' on the right. In the center of the page is a white rectangular form. At the top of the form is a red heart icon, followed by the title 'Physician Portal' in bold. Below the title is a subtitle: 'Sign in to view your patients and their data.' There are two links: 'Sign In' and 'Register' (which is underlined and highlighted in red). Below these links are four input fields: 'Full Name', 'Email' (with the placeholder text 'physician@example.com'), 'Password', and 'Confirm Password'. At the bottom of the form is a red button with the text 'Create Account' in white.

Lessons Learned

1. Mastering the Express framework enabled me to structure a robust RESTful backend, specifically by wiring authentication guards and exposing clean measurement endpoints that the frontend could consume without requiring extra adapters.
2. Deploying to AWS offered practical experience in configuring environment variables, process management, and remote logging, ensuring the application runs reliably outside a local development environment.
3. Careful data modeling—specifically decoupling users, devices, measurements, and physicians—clarified system responsibilities and made debugging ownership issues significantly more manageable for me as a student developer.
4. Embedding physician roles in JWTs and enforcing them via middleware demonstrated that reliable access control is attainable through modest code, offering a lightweight alternative to adopting complex third-party identity systems.
5. Normalizing analytic queries to UTC boundaries before processing prevented mismatches between local laptop clocks and server time, ensuring that chart visualizations remained consistent and trustworthy during demos.
6. Persisting physician portal state in browser storage while reconciling it with fresh API responses taught me to treat local caches as provisional data, ensuring the user interface remains coherent after page refreshes.
7. Implementing both client-side and server-side password validation reinforced the importance of layered security, guiding users toward stronger credentials while strictly enforcing hashing and complexity at the API level.

Challenges

1. Synchronizing the bidirectional relationship between patients and physicians was difficult because the database does not enforce foreign key integrity automatically. Resolution: I implemented dedicated backend logic to ensure that when a user selects a doctor, the physician's patient list is updated simultaneously to reflect the change.
2. Handling the registration of new IoT devices was problematic because devices might send data before being manually registered by the user. Resolution: I developed logic within the data ingestion API that checks if a device ID exists and automatically creates a new device record linked to the user if it is missing.
3. Managing distinct authentication flows for two different types of users (Patients and Physicians) within the same application structure was confusing. Resolution: I established separate database models and distinct frontend entry points to completely isolate the logic and permissions for each role.
4. Rendering the dashboard charts correctly required converting raw database timestamps into user-friendly time labels. Resolution: I implemented a server-side formatting utility to transform the ISO date objects into readable time strings before sending the response to the frontend.
5. Ensuring the application loaded all necessary data (user profile, devices, and metrics) before the UI appeared caused visual "flickering" issues. Resolution: I utilized parallel promise execution within an initialization function to fetch all required data endpoints simultaneously, ensuring the state was fully prepared before revealing the application interface.

Contribution

Task	Yutong Liu	Kaiyuan Deng	Zhuo Wang
System Implementation	Frontend Built responsive UI with Tailwind CSS; Implemented Chart.js visualization and DOM logic.	Backend Developed RESTful APIs, MongoDB schemas, and JWT authentication logic.	Hardware Integrated sensors; Implemented device-side logic for data collection and API transmission.
Documentation & Demos	Collaborated on the report and document.	Collaborated on the report and document.	
Total Contribution	33.3%	33.3%	33.3%